SPENDWISE

INTRODUCATION

Project Title: SPENDWISE- A Simple Backend Learning Project

- Team ID :SWTID-2026-2725
- Team Leader : Nisha S
  - Email : sureshnisha2821@gmail.com
- Team Members :
  1. Nikitha U
     - Email : nikithauma1234@gmail.com
  2. Nithya Shree B
     - Email : b.nithyashree25@gmail.com
  3. Padma Priya M
     - Email : padma.priya.m24@gmail.com
  4. Padma R
     - Padmaram2251@gmail.com

Project Overview

SpendWise is a simple backend application developed to help beginners understand the fundamentals of backend development using modern technologies such as **Node.js, Express.js, MongoDB, and JWT authentication**. The main purpose of this project is to demonstrate how a backend system works and how secure APIs can be created for managing user data.

Key Objectives:

- **Understand Backend Architecture**
  To demonstrate how backend components such as models, routes, controllers, and middleware work together.
- **Implement Secure User Authentication**
  To develop a secure registration and login system using JWT (JSON Web Token) for user authentication.
- **Perform CRUD Operations**
  To allow users to Create, Read, Update, and Delete transaction records through REST APIs.
- **Integrate MongoDB Database**
  To store and manage user and transaction data efficiently using MongoDB with Mongoose.

Target Users

- **Beginner Backend Developers**
  Individuals who are new to backend development and want to learn how to build APIs using Node.js and Express.
- **Students Learning Web Development**
  College students who are studying web technologies and want practical experience with backend projects.
- **Freshers Preparing for Technical Interviews**
  Graduates or job seekers who want to strengthen their backend development knowledge and project experience.
- **Developers Learning MongoDB and JWT**
  Learners who want to understand how MongoDB databases and JWT authentication work in real applications.

Project Instruction

The **SpendWise** project is a backend application developed using **Node.js and Express.js**. It is designed to help learners understand how to build secure APIs and manage data using a database.

Quick Start:

npm init

npm install

npm run dev

Steps to Create the Project:

1. **Create a Project Folder**
   First, create a new folder for the backend project and open it in a code editor such as Visual Studio Code.
2. **Initialize the Node.js Project**
   Open the terminal in the project folder and run the command npm init to create a new Node.js project and generate a package.json file.
3. **Install Required Dependencies**
   Install the necessary packages like Express, Mongoose, JSON Web Token, Bcrypt, Cors, and Dotenv using npm.
4. **Create Project Folder Structure**
   nize the project by creating folders such as models, routes, controllers, middleware, and config to manage the code properly.

Software Requirements

Software	Purpose
Node.js	JavaScript runtime for backend
MongoDB	NoSQL database for data storage
npm	Package manager
Git	Version control system
Code Editor	VS Code or similar

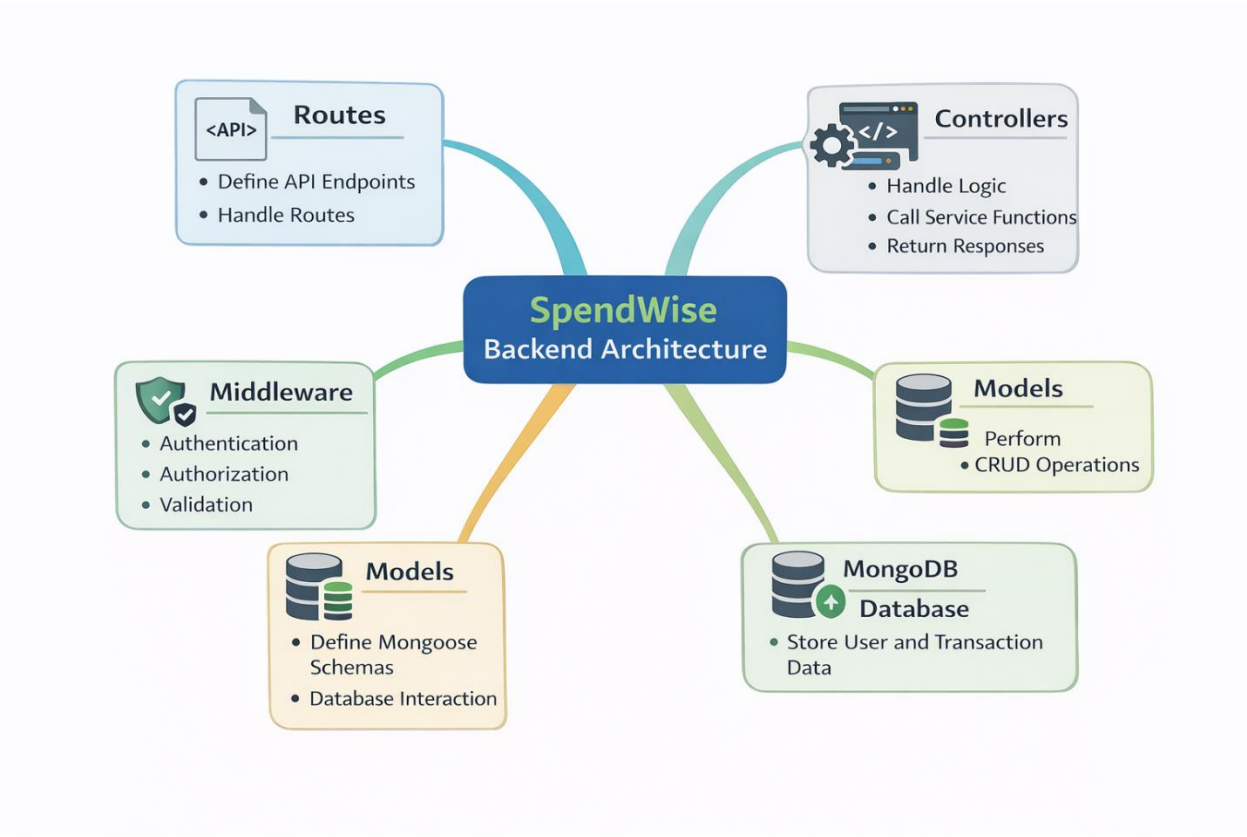
Knowledge Requirements

- Basic JavaScript – Simple knowledge of JavaScript.
- Node.js – Basic idea about Node.js.
- Express – Used to create APIs.
- MongoDB – Used to store data.
- JWT – Used for user login security.
- API Methods – Basic idea of GET, POST, PUT, DELETE**.**

Thumbnail for Project

A simple thumbnail that shows the technologies used in the project like **Node.js, Express, MongoDB, JWT, and CRUD APIs**.
It helps to quickly understand that the project is a **backend application**.

Project Architecture Image



Technical Architecture

Backend Architecture (Node.js + Express)

1. **Routes – API paths (ex: login, register).**
2. **Controllers – Handle the work / logic.**
3. **Models – Database structure (User, Transaction)**
4. **Middleware – Check JWT token (security).**
5. **Config – Connect to MongoDB.**

Database Architecture (MongoDB)

- **Users – stores user details (name, email, password)**

- **Transactions – stores money records (title, amount, userId)**
- **Relationship: 1 User → Many Transaction**

Features

Features	DESCRIPTION
User Authentication	Users can register and login using JWT.
CRUD Operations	Users can create, read, update, and delete transactions.
Protected Routes	JWT middleware protects private APIs.
MongoDB Integration	Data is stored and retrieved from MongoDB.

Roles and Responsibilities

Student / Developer

- Write backend code
- Create APIs
- Test APIs
- Upload code to GitHub
-

System (Backend)

- Authenticate users (JWT)
- Handle CRUD operations
- Store data in MongoDB
- Protect private routes

BACKEND DEVELOPMENT

- Setup Express Server
  Create server.js.
- Create .env file
  Add PORT and Database URL.

- Configure middleware (cors, express.json)
- User Authentication:
  - Create Register and Login routes
  - Generate JWT token
  - Protect routes using middleware

- Define API Routes:
  - Auth routes

In this step, you create **authentication routes** for user **registration and login** in the backend application. These routes are created using **Express.js** in a **Node.js** server.

- Transaction CRUD routes

```js
const express = require("express");
const dotenv = require("dotenv");
const cors = require("cors");
const connectDB = require("./config/db");

dotenv.config();

// Connect to MongoDB
connectDB();

const app = express();

// Middleware
app.use(cors());
app.use(express.json());

// Test route
app.get("/", (req, res) => {
    res.send("Spendwise API is running...");
});

// Routes
app.use("/api/auth", require("./routes/authRoutes"));
```

- Transaction schema

```js
// Test route
app.get("/", (req, res) => {
    res.send("Spendwise API is running...");
});

// Routes
app.use("/api/auth", require("./routes/authRoutes"));
app.use("/api/transactions", require("./routes/transactionRoutes"));

const PORT = process.env.PORT || 5000;

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

DATABASE DEVELOPMENT

Configure MongoDB

- Use **MongoDB Atlas** or **Local MongoDB**.
- Save the **database URL** in the .env file.
- Connect the database using **Mongoose**.

Create Schema and Models

User Model:

- **name** – User name
- **email** – User email
- **password** – User password

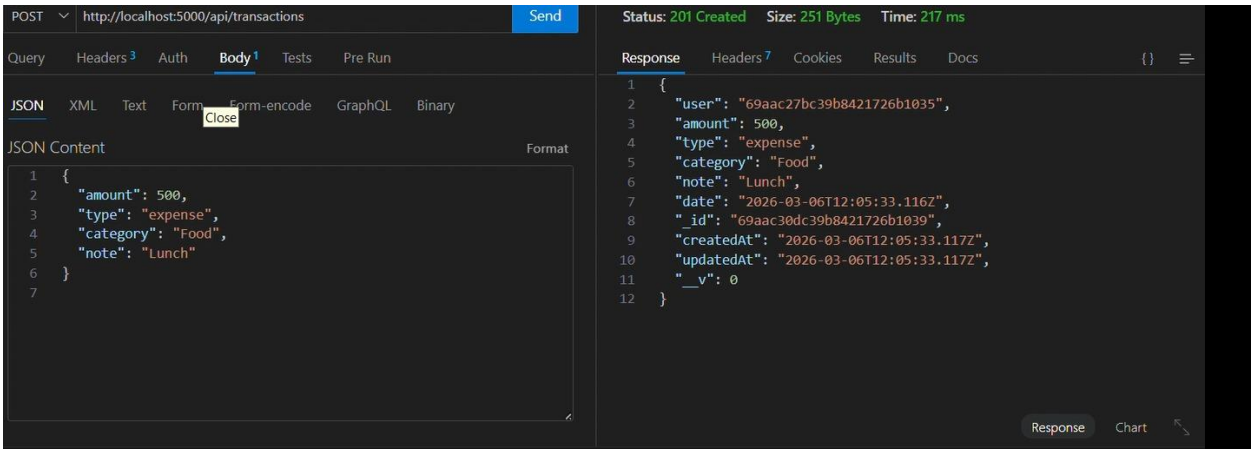
Data / Transaction Model:

1. **userId** – Which user made the transaction
2. **title / description** – Transaction details
3. **amount** – Money value

PROJECT EXECUTION

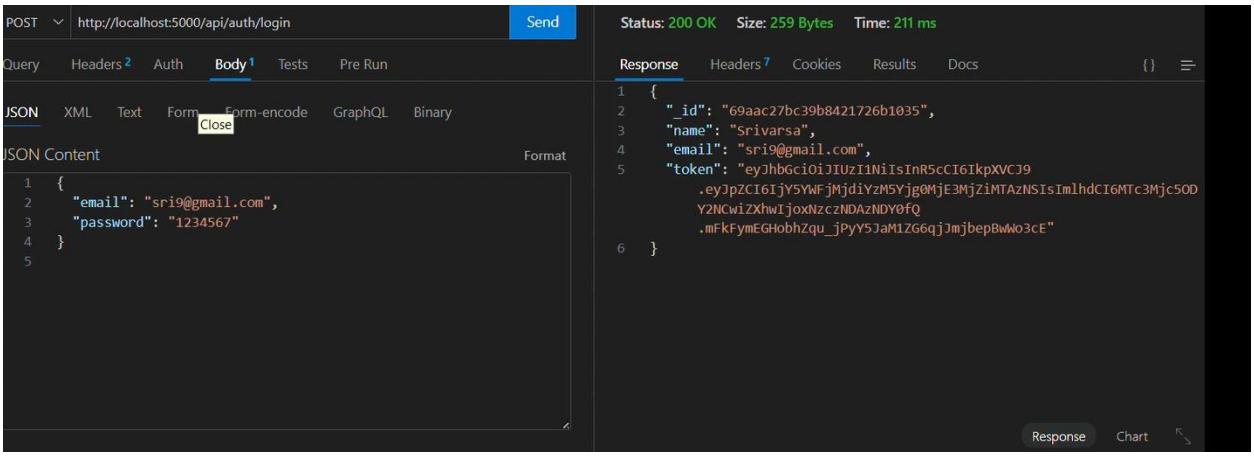
1. Star MongoDB
   Run MongoDB database.
2. Run backend server
   Start the Node.js server

3.Test Register API
   Create a new user account.



4. Test Login API

   Login and get **JWT token**.



5. Copy JWT token

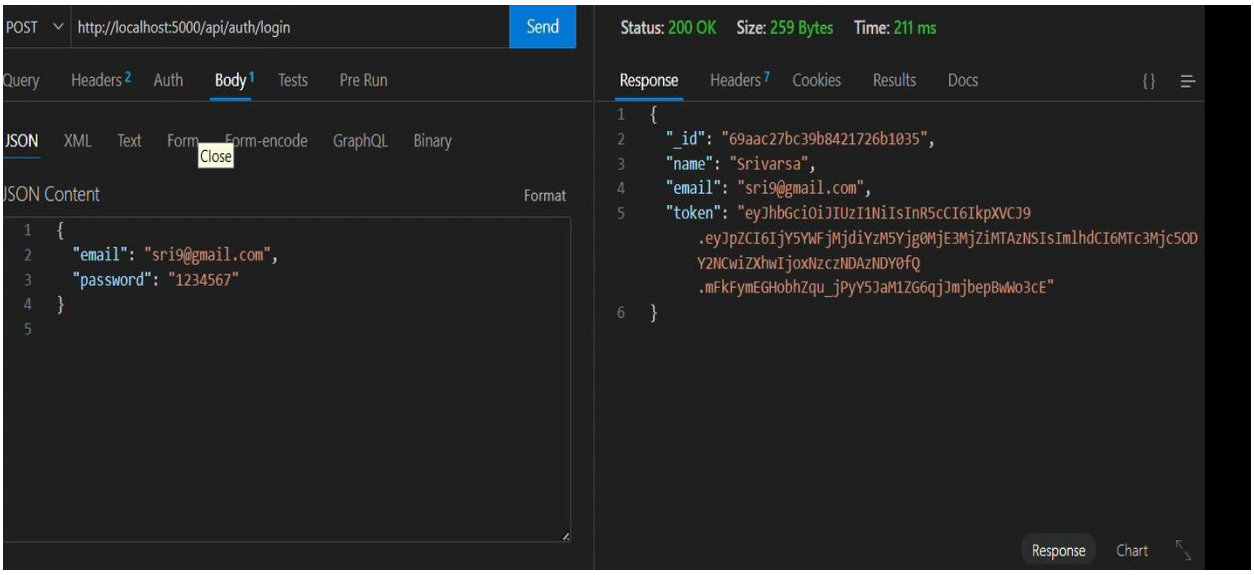
   Access protected APIs and perform **Create, Read, Update, Delete transactions**.

- 1.User **logs in**
- 2.System gives **JWT token**

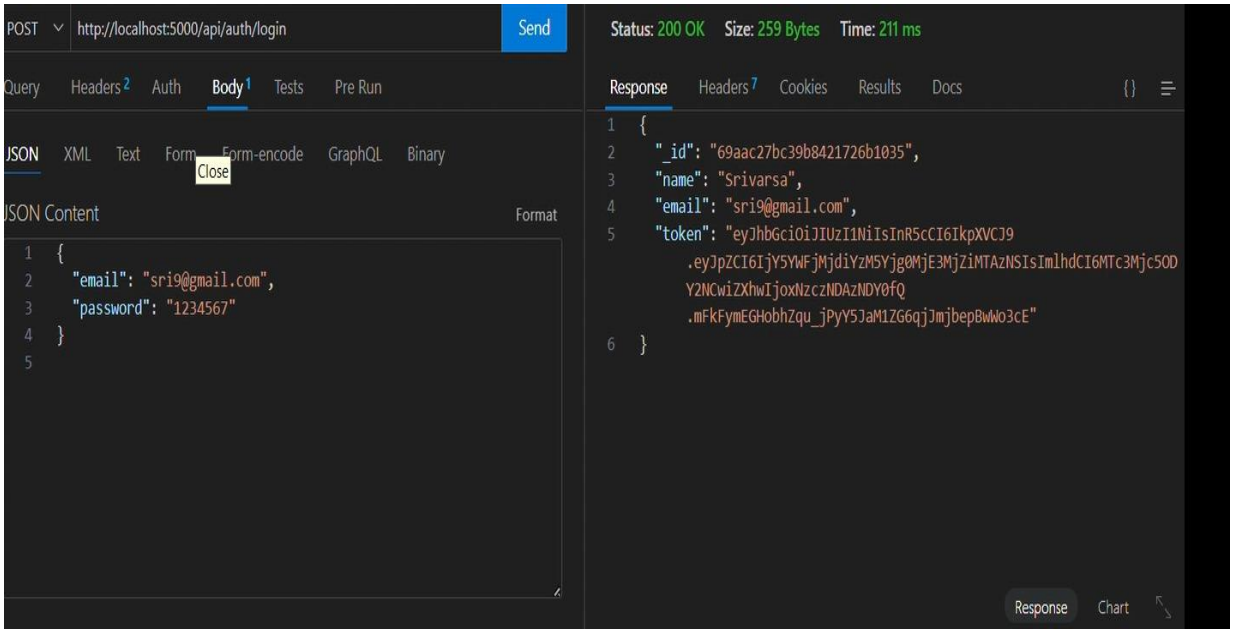
  Example:
- 3.eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
  - When the user wants to **create a transaction**, they send the token.

OUTPUT SCREENSHOT

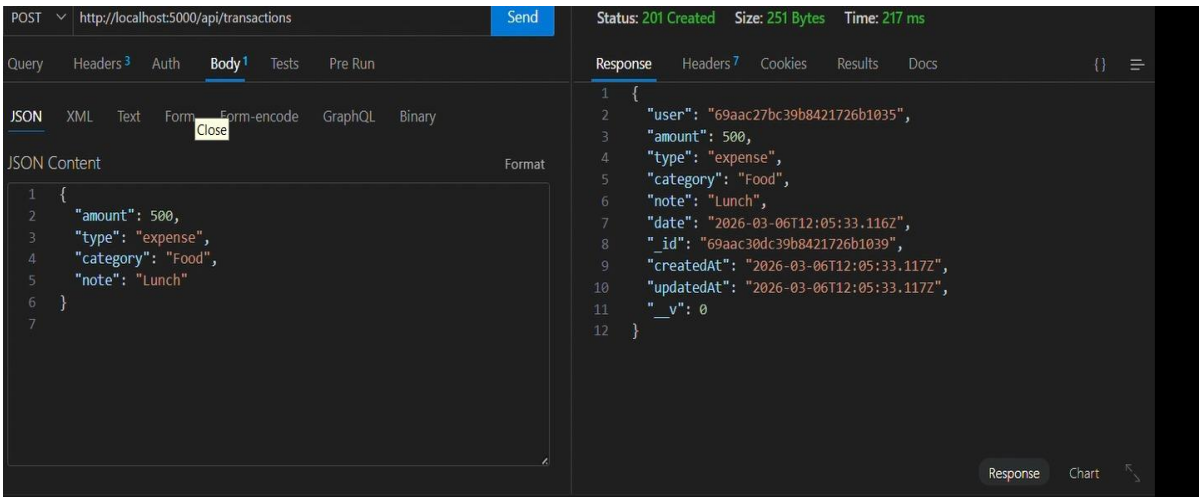
1. Register API



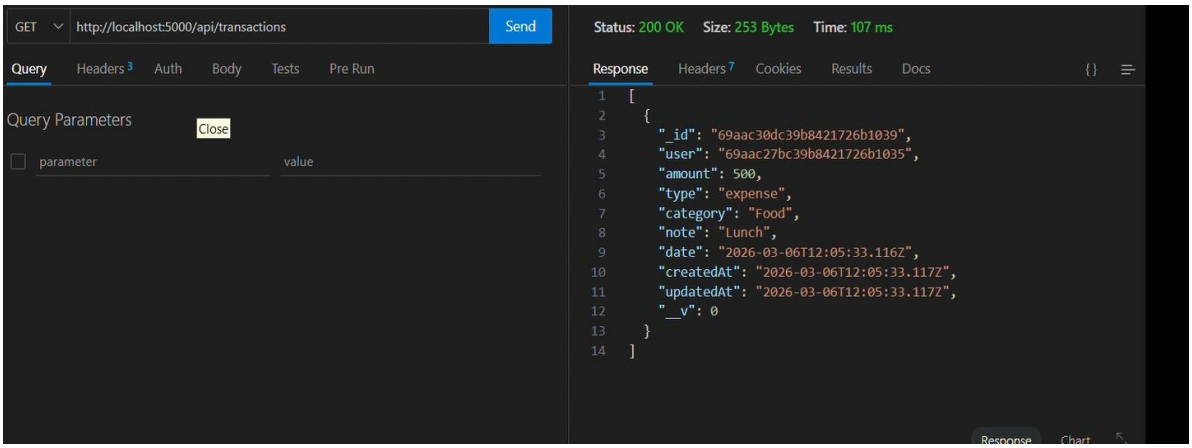
2. Login API



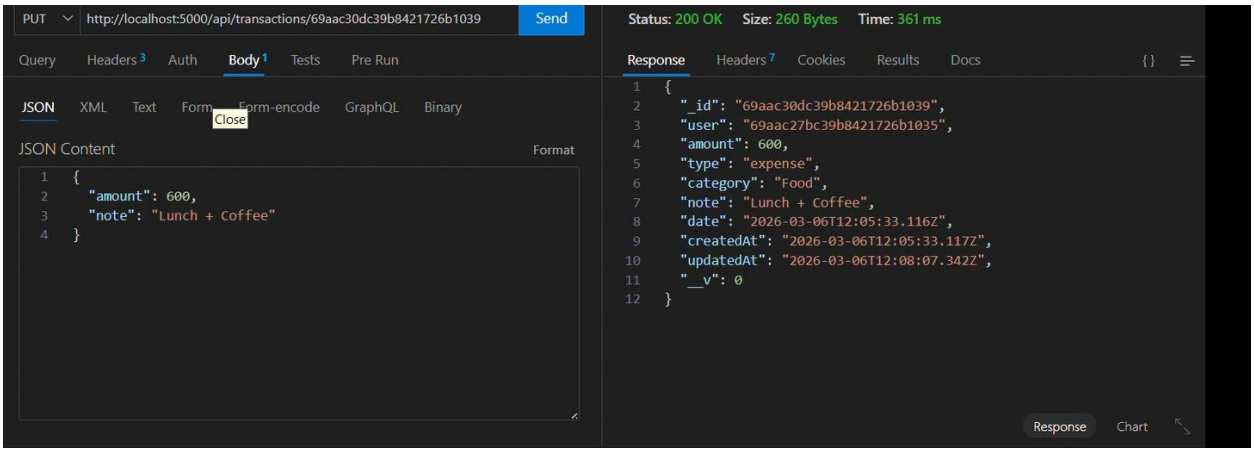
3. Create Transaction



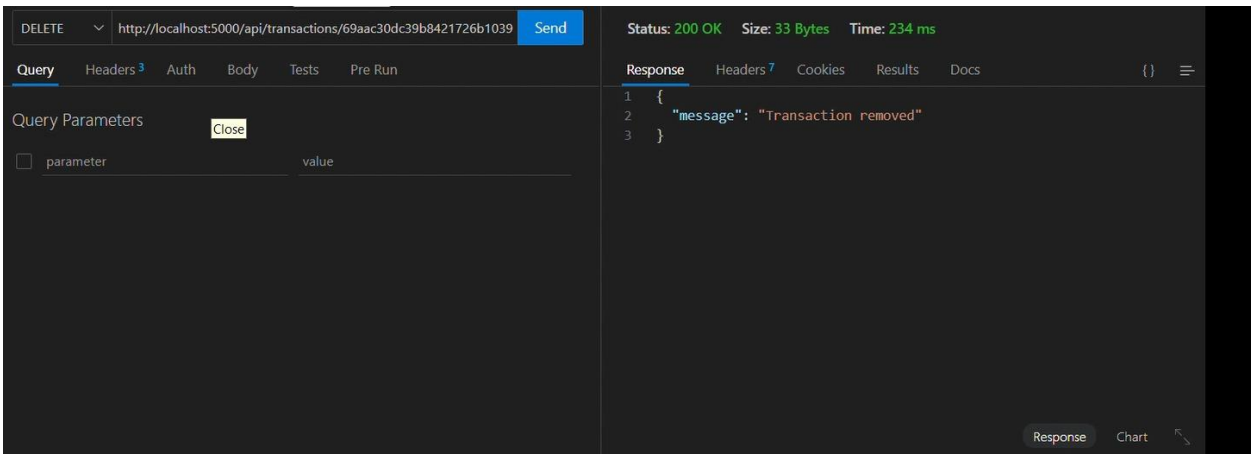
4. Get information



5. Update API


PUT
http://localhost:5000/api/transactions/69aac30dc39b8421726b1039
Send
Status: 200 OK
Size: 260 Bytes
Time: 361 ms
Query
Headers 3
Auth
Body 1
Tests
Pre Run
Response
Headers 7
Cookies
Results
Docs
JSON
XML
Text
Form
Form-encode
GraphQL
Binary
Close
JSON Content
Format
{
"amount": 600,
"note": "Lunch + Coffee"
}
{
"_id": "69aac30dc39b8421726b1039",
"user": "69aac27bc39b8421726b1035",
"amount": 600,
"type": "expense",
"category": "Food",
"note": "Lunch + Coffee",
"date": "2026-03-06T12:05:33.116Z",
"createdAt": "2026-03-06T12:05:33.117Z",
"updatedAt": "2026-03-06T12:08:07.342Z",
"__v": 0
}
Response
Chart

6. Delete API


DELETE
http://localhost:5000/api/transactions/69aac30dc39b8421726b1039
Send
Status: 200 OK
Size: 33 Bytes
Time: 234 ms
Query
Headers 3
Auth
Body
Tests
Pre Run
Response
Headers 7
Cookies
Results
Docs
Query Parameters
Close
parameter
value
{
"message": "Transaction removed"
}
Response
Chart